

1. Visão Geral da Arquitetura

O pipeline segue uma arquitetura em três camadas, orquestrada pelo Prefect e transformada pelo dbt, seguindo o padrão medalhão (Bronze / Silver / Gold) amplamente adotado em Data Lakehouses:

- Bronze (raw): dados brutos da Receita Federal (Empresas, Estabelecimentos, Sócios, Simples Nacional, CNAE), carregados sem transformação pela camada de extração.
- Silver (staging): padronização básica, tipagem, parsing de datas/decimais no formato brasileiro, decodificação de domínios (situação cadastral, porte).
- Gold (marts): modelo dimensional consumido por BI/analytics, dimensões (dim_empresa, dim_cnae) e uma tabela fato incremental (fct_empresas_ativas).

Em produção na GCP, essa arquitetura se traduziria da seguinte forma:

Camada local (DuckDB)	Equivalente em produção (GCP)	Observação
Extração (Prefect task)	Cloud Composer (Airflow) ou Prefect Cloud + Cloud Run	Orquestração gerenciada, com scheduling e alertas
raw (schema DuckDB)	BigQuery dataset `raw` (ou GCS + BigQuery external tables)	Armazenamento bruto, sem transformação
staging / intermediate (views)	BigQuery dataset `staging`, materializações view	Sem custo de armazenamento adicional
marts (tables)	BigQuery dataset `marts`, tabelas particionadas/clusterizadas	Camada consumida por BI (Looker, Metabase, etc.)
snapshot SCD2	BigQuery table com estratégia de merge incremental	Histórico de capital_social

2. Estratégia de Materialização e Orquestração

2.1 Materializações por camada

- Staging: views, não há custo de storage e sempre refletem o dado mais recente da camada raw. Como o volume por execução é baixo (apenas ponteiros para a raw), o custo de computação em tempo de query é desprezível.
- Intermediate: views, mesma lógica de staging, usadas para reuso de lógica de negócio (agregação de sócios, hierarquia de CNAE) sem duplicar dado em disco.
- Marts (dimensões): tabelas materializadas, atualizadas a cada execução completa, pois o volume de empresas/CNAEs é relativamente estável e cabe em uma reconstrução full a cada carga.
- Marts (fato): incremental com merge por unique_key (cnpj_basico), evita reprocessar o histórico completo (+50M registros em produção) a cada execução.

2.2 Orquestração (Prefect → dbt → BigQuery)

O Flow desenvolvido (pipeline_flow.py) já incorpora práticas de resiliência que se mantêm válidas em produção:

- Retries automáticos configurados por task (2 tentativas na extração, 1 na transformação), com backoff fixo, absorve falhas transitórias de rede/lock sem intervenção manual.
- Separação de responsabilidades por task: falha na extração não mascara falha em teste de qualidade, cada etapa reporta seu próprio status no log do Prefect.
- dbt test como gate de qualidade: em produção, um resultado de erro (não apenas warning) no dbt test deve interromper a publicação da camada gold para consumo por BI, evitando propagar dado ruim a stakeholders.

- Idempotência: a estratégia incremental via merge/unique_key garante que reexecuções do mesmo lote não dupliquem registros.

3. Estratégia de Partitioning no BigQuery

Partitioning é a alavanca de maior impacto em custo no BigQuery, pois o billing é cobrado por bytes lidos, particionar corretamente reduz drasticamente o volume de dados escaneado por query.

3.1 fct_empresas_ativas

- Partição sugerida: DATE_TRUNC(data_situacao_cadastral, MONTH), usando partição por coluna de data (não por data de ingestão), pois a maioria das consultas de negócio filtra por período de alteração cadastral ("empresas que mudaram de status em determinado mês").
- Alternativa complementar: partição por data de carga (_carregado_em) caso o consumo predominante seja "snapshot mais recente", comum em dashboards operacionais.
- Retenção: partition expiration de 24-36 meses para dados operacionais, mantendo o histórico completo apenas na camada raw (armazenamento em GCS, mais barato que BigQuery storage).

3.2 snap_empresas_capital_social (snapshot SCD2)

- Partição por dbt_valid_from (mensal), consultas de auditoria/histórico tipicamente filtram por período de vigência.

3.3 Tabelas de staging/dimensão

- dim_empresa e dim_cnae: volume relativamente pequeno (dezenas de milhões de linhas no máximo), particionamento não traz ganho relevante; priorizar clustering (seção 4).
- stg_estabelecimentos: se materializada como tabela em algum cenário de troubleshooting, particionar por data_inicio_atividade (mensal).

4. Estratégia de Clustering

Clustering reordena fisicamente os dados dentro de cada partição, permitindo que o BigQuery pule blocos inteiros de dados (block pruning) quando a query filtra pelas colunas de cluster.

Tabela	Colunas de cluster (ordem)	Justificativa
fct_empresas_ativas	uf, cnae_fiscal_principal, cnpj_basico	Consultas de negócio predominantemente filtram/agrupam por UF e setor (CNAE); cnpj_basico como 3º nível acelera lookups pontuais
dim_empresa	uf, porte_empresa_desc	Segmentações de BI mais comuns são por região e porte da empresa
snap_empresas_capital_social	cnpj_basico	Consultas de auditoria tipicamente buscam o histórico de uma empresa específica

- Observação sobre ordem das colunas de cluster: o BigQuery usa a ordem declarada para otimizar prefixos de filtro, colocar a coluna de maior seletividade/uso mais frequente primeiro (UF, no caso de fct_empresas_ativas) maximiza o benefício mesmo quando apenas parte das colunas de cluster é usada no filtro.

5. Estimativa Conceitual de Impacto em Custo e Performance

O BigQuery cobra por volume de dados escaneado (on-demand) ou por slots reservados (capacity). Nas duas modalidades, reduzir bytes lidos por query reduz custo/slots consumidos proporcionalmente. Estimativas abaixo são conceituais (ordem de grandeza), não uma auditoria de custo real, já que dependem do volume final de dados e do padrão de consulta em produção.

5.1 Efeito do Partitioning

Considerando uma tabela `fcf_empresas_ativas` com histórico de ~36 meses de alterações cadastrais, sem partição, uma consulta típica de BI ("empresas ativas nos últimos 30 dias") escanearia 100% da tabela. Com partição mensal por `data_situacao_cadastral`, a mesma consulta lê apenas ~1/36 do volume, uma redução da ordem de 97% nos bytes escaneados, e portanto no custo dessa query.

5.2 Efeito do Clustering

Dentro de uma única partição mensal, uma consulta filtrando por UF específica (ex: "empresas ativas em SP em janeiro/2026") sem clustering ainda leria a partição inteira. Com clustering por UF como primeira coluna, o BigQuery pode podar blocos referentes às demais 26 UFs, reduzindo o volume lido dentro da partição em uma proporção próxima ao inverso do número de UFs filtradas, na prática, algo entre 80-95% de redução adicional para filtros de UF única, dependendo da distribuição real dos dados.

5.3 Efeito combinado

Combinando partition pruning (mensal) e clustering (UF/CNAE), uma consulta segmentada por período e região pode reduzir os bytes escaneados em mais de 99% frente a uma tabela sem otimização, o que se traduz diretamente em menos slots consumidos (capacity pricing) ou menor custo por TB escaneado (on-demand pricing).

5.4 Recomendações adicionais de FinOps

- Materialized Views para agregações recorrentes de dashboard (ex: contagem de empresas ativas por UF/CNAE), evitando reprocessar a tabela fato a cada refresh de BI.
- BI Engine ou cache de resultados para consultas repetidas do mesmo dashboard dentro de uma janela curta de tempo.
- Monitoramento de custo via `INFORMATION_SCHEMA.JOBS`, com alertas para queries que escaneiam volume anômalo (possível ausência de filtro de partição).
- `require_partition_filter=true` nas tabelas particionadas, forçando que toda query declare o filtro de partição, evita full scans acidentais por erro de usuário/BI tool.

6. Onboarding de Novos Colaboradores

Passo a passo para um novo colaborador do time de dados começar a contribuir com este pipeline:

- 1. Clonar o repositório e revisar a estrutura de pastas: `dbt/` (transformação), `flows/` (orquestração Prefect), `data/` (amostras e scripts de carga).
- 2. Seguir o `README.md` na raiz do projeto para setup do ambiente local (Python venv, dependências, DuckDB).
- 3. Rodar o pipeline localmente (`python flows/pipeline_flow.py`) para validar o ambiente antes de qualquer alteração.
- 4. Familiarizar-se com a convenção de camadas: modelos em `staging/` nunca aplicam regra de negócio (apenas tipagem/padronização); regras de negócio vivem em `intermediate/` e `marts/`.
- 5. Toda nova coluna/regra de negócio deve vir acompanhada de teste dbt correspondente (genérico ou customizado via macro), ver `macros/test_valor_positivo.sql` como referência de teste customizado.
- 6. Em produção (BigQuery), respeitar as convenções de partitioning/clustering descritas nas seções 3 e 4 deste documento ao criar novas tabelas na camada `marts`.

7. Link do Projeto

Código-fonte completo (dbt + Prefect + testes + macros + snapshot SCD2):
<https://github.com/Ynnayron/Teste-Engenharia-de-Dados>

O repositório inclui: projeto dbt em camadas (staging/intermediate/marts), 4 macros Jinja customizadas, 35 testes dbt (genéricos e customizados), snapshot SCD2 para histórico de capital social, Prefect Flow de orquestração, e amostras reais de dados da Receita Federal para reprodução local.